BLE na iOS — Zwięzły Przewodnik (Core Bluetooth)

> Swift 5.9+, iOS 13+. Core Bluetooth jest delegate-based — `async/await` nie jest natywnie wspierane przez framework.

Spis treści

1. Podstawy: Central vs Peripheral, GATT
2. Konfiguracja projektu
3. CBCentralManager — rola Central
4. Skanowanie i Discovery
5. Łączenie z Peripheral
6. GATT: Services & Characteristics
7. Odczyt, Zapis, Notifications
8. CBPeripheralManager — rola Peripheral
9. Background Modes
10. State Restoration
11. Best Practices & Gotchas

---

1. Podstawy

Role urządzeń

Rola	Klasa	Opis
Central	`CBCentralManager`	Skanuje, łączy się, czyta/pisze dane
Peripheral	`CBPeripheralManager`	Rozgłasza się (advertising), obsługuje zapytania

iPhone może pełnić **obie role jednocześnie**.

Hierarchia GATT

```
Peripheral
└── Service (UUID, np. Heart Rate: 0x180D)
```

```
└── Characteristic (UUID, wartość + właściwości)
    └── Descriptor (metadane, np. CCCD 0x2902)
```

- **UUID**: 16-bit (standardowe Bluetooth SIG) lub 128-bit (własne)
- **CCCD** (Client Characteristic Configuration Descriptor) — włącza notifications/indications

---

2. Konfiguracja projektu

Info.plist — wymagane klucze

```
<!-- Zawsze wymagane -->
<key>NSBluetoothAlwaysUsageDescription</key>
<string>Aplikacja używa Bluetooth do komunikacji z urządzeniem.</
        string>

<!-- Dla Background modes (opcjonalne) -->
<key>UIBackgroundModes</key>
<array>
    <string>bluetooth-central</string>
    <string>bluetooth-peripheral</string>
</array>
```

> **Uwaga:** Bez `NSBluetoothAlwaysUsageDescription` aplikacja crashuje na starcie na iOS 13+.

Import

```
import CoreBluetooth
```

---

3. CBCentralManager

Inicjalizacja

```
class BLEManager: NSObject, CBCentralManagerDelegate {
    var centralManager: CBCentralManager!

    override init() {
        super.init()
        // Zawsze inicjalizuj na main queue lub dedykowanej queue
        centralManager = CBCentralManager(delegate: self,
        queue: .main)
        // Z opcjami dla State Restoration:
        // centralManager = CBCentralManager(
        //     delegate: self,
        //     queue: .main,
```

```
        //     options:
        [CBCentralManagerOptionRestoreIdentifierKey:
        "myAppRestoreID"]
        // )
    }

    // WYMAGANE — sprawdź stan ZANIM zaczniesz skanować
    func centralManagerDidUpdateState(_ central:
        CBCentralManager) {
        switch central.state {
        case .poweredOn:
            print("BLE gotowe")
            // Tu możesz zacząć skanować
        case .poweredOff:
            print("Bluetooth wyłączony")
        case .unauthorized:
            print("Brak uprawnień")
        case .unsupported:
            print("BLE nieobsługiwane")
        case .resetting:
            print("BLE resetuje się")
        case .unknown:
            print("Stan nieznany")
        @unknown default:
            break
        }
    }
}
```

Stany `CBManagerState`

```
unknown → resetting → unsupported
                    → unauthorized
                    → poweredOff ↔ poweredOn
```

---

4. Skanowanie i Discovery

Rozpoczęcie skanowania

```
// Skanuj wszystkie urządzenia
centralManager.scanForPeripherals(withServices: nil, options: nil)

// Skanuj tylko urządzenia z określoną usługą (bardziej efektywne
        energetycznie)
let serviceUUID = CBUUID(string: "180D") // Heart Rate Service
centralManager.scanForPeripherals(withServices: [serviceUUID],
        options: [
```

```
    CBCentralManagerScanOptionAllowDuplicatesKey: false // true =
        każdy packet osobno
])
```

Odbiór wyników

```
func centralManager(
    _ central: CBCentralManager,
    didDiscover peripheral: CBPeripheral,
    advertisementData: [String: Any],
    rssi RSSI: NSNumber
) {
    let name = peripheral.name ?? "Nieznane"
    let rssiValue = RSSI.intValue // w dBm, np. -55 = silny sygnał

    // Ważne klucze advertisementData:
    let localName =
        advertisementData[CBAdvertisementDataLocalNameKey] as?
        String
    let serviceUUIDs =
        advertisementData[CBAdvertisementDataServiceUUIDsKey] as?
        [CBUUID]
    let txPower =
        advertisementData[CBAdvertisementDataTxPowerLevelKey] as?
        Int
    let manufacturerData =
        advertisementData[CBAdvertisementDataManufacturerDataKey]
        as? Data
    let isConnectable =
        advertisementData[CBAdvertisementDataIsConnectable] as?
        Bool

    print("Znaleziono: \(name), RSSI: \(rssiValue) dBm")

    // Zachowaj referencję! Peripheral jest ARC-managed.
    self.discoveredPeripheral = peripheral
}
```

Zatrzymanie skanowania

```
centralManager.stopScan() // Zawsze zatrzymuj po znalezieniu
        urządzenia!
```

> **Gotcha:** Skanowanie z `nil` serviceUUIDs w tle jest **blokowane** przez iOS.

---

5. Łączenie z Peripheral

Connect / Disconnect

```
// Łączenie
centralManager.connect(peripheral, options: nil)
// Opcja: [CBConnectPeripheralOptionNotifyOnConnectionKey: true] –
        powiadomienie gdy w tle

// Rozłączanie
centralManager.cancelPeripheralConnection(peripheral)
```

Callbacki połączenia

```
// Sukces
func centralManager(_ central: CBCentralManager, didConnect
        peripheral: CBPeripheral) {
    print("Połączono z \(peripheral.name ?? "?")")
    peripheral.delegate = self  // Ustaw delegata TUTAJ
    peripheral.discoverServices(nil) // nil = wszystkie usługi
        (kosztowne)
    // Lepiej: peripheral.discoverServices([serviceUUID])
}

// Błąd połączenia
func centralManager(
    _ central: CBCentralManager,
    didFailToConnect peripheral: CBPeripheral,
    error: Error?
) {
    print("Błąd połączenia: \(error?.localizedDescription ??
        "nieznany")")
}

// Rozłączenie
func centralManager(
    _ central: CBCentralManager,
    didDisconnectPeripheral peripheral: CBPeripheral,
    error: Error?
) {
    if let error = error {
        print("Nieoczekiwane rozłączenie: \(error)")
        // Możesz ponowić połączenie
        centralManager.connect(peripheral, options: nil)
    } else {
        print("Rozłączono poprawnie")
    }
}
```

Ponowne łączenie ze znanym Peripheral (po restarcie)

```
// Pobierz peripheral po UUID (nie skanuj ponownie)
let knownUUID = UUID(uuidString: "12345678-...")!
let peripherals =
        centralManager.retrievePeripherals(withIdentifiers:
        [knownUUID])
if let peripheral = peripherals.first {
    centralManager.connect(peripheral, options: nil)
}

// Lub pobierz podłączone systemy z daną usługą
let connected =
        centralManager.retrieveConnectedPeripherals(withServices:
        [serviceUUID])
```

---

6. GATT: Services & Characteristics

Odkrywanie Services

```
// CBPeripheralDelegate
func peripheral(_ peripheral: CBPeripheral, didDiscoverServices
        error: Error?) {
    guard error == nil, let services = peripheral.services else {
        return }

    for service in services {
        print("Usługa: \(service.uuid)")
        // Odkryj charakterystyki dla każdej usługi
        peripheral.discoverCharacteristics(nil, for: service)
        // Lepiej: peripheral.discoverCharacteristics([charUUID],
        for: service)
    }
}
```

Odkrywanie Characteristics

```
func peripheral(
    _ peripheral: CBPeripheral,
    didDiscoverCharacteristicsFor service: CBService,
    error: Error?
) {
    guard error == nil, let characteristics =
        service.characteristics else { return }

    for char in characteristics {
        print("Charakterystyka: \(char.uuid), właściwości: \
        (char.properties)")

        // Sprawdzanie właściwości
```

```
        if char.properties.contains(.read) { /* można czytać */ }
        if char.properties.contains(.write) { /* zapis z
        potwierdzeniem */ }
        if char.properties.contains(.writeWithoutResponse) { /*
        zapis bez potwierdzenia, szybszy */ }
        if char.properties.contains(.notify) { /* subskrypcja
        powiadomień */ }
        if char.properties.contains(.indicate) { /* jak notify,
        ale z ACK */ }
    }
}
```

Właściwości `CBCharacteristicProperties`

Właściwość	Opis
`.read`	Jednorazowy odczyt wartości
`.write`	Zapis z potwierdzeniem (ACK)
`.writeWithoutResponse`	Zapis bez ACK — szybszy, brak gwarancji dostarczenia
`.notify`	Peripheral wysyła dane bez pytania (bez ACK)
`.indicate`	Jak notify, ale z ACK od Central
`.broadcast`	Wartość w advertisement packet

7. Odczyt, Zapis, Notifications

Odczyt

```
peripheral.readValue(for: characteristic)

func peripheral(
    _ peripheral: CBPeripheral,
    didUpdateValueFor characteristic: CBCharacteristic,
    error: Error?
) {
    guard error == nil, let data = characteristic.value else {
        return }
    // data to Data — dekoduj według protokołu urządzenia
    let value = data.withUnsafeBytes { $0.load(as: UInt8.self) }
    print("Wartość: \(value)")
}
```

Zapis

```
let data = Data([0x01, 0x02])

// Z potwierdzeniem (wolniejszy, pewny)
```

```swift
peripheral.writeValue(data, for: characteristic,
        type: .withResponse)

// Bez potwierdzenia (szybki, bez gwarancji)
peripheral.writeValue(data, for: characteristic,
        type: .withoutResponse)

// Callback tylko dla .withResponse
func peripheral(
    _ peripheral: CBPeripheral,
    didWriteValueFor characteristic: CBCharacteristic,
    error: Error?
) {
    if let error = error {
        print("Błąd zapisu: \(error)")
    } else {
        print("Zapis potwierdzony")
    }
}

// Sprawdzenie gotowości dla .withoutResponse
func peripheralIsReady(toSendWriteWithoutResponse peripheral:
        CBPeripheral) {
    // Można teraz wysłać następną porcję danych
}
```

Notifications / Indications

```swift
// Włącz
peripheral.setNotifyValue(true, for: characteristic)

// Wyłącz
peripheral.setNotifyValue(false, for: characteristic)

// Callback potwierdzenia subskrypcji
func peripheral(
    _ peripheral: CBPeripheral,
    didUpdateNotificationStateFor characteristic:
        CBCharacteristic,
    error: Error?
) {
    if characteristic.isNotifying {
        print("Notyfikacje włączone dla \(characteristic.uuid)")
    }
}

// Odbiór danych — ten sam callback co przy readValue:
// func peripheral(_:didUpdateValueFor:error:)
```

MTU — maksymalny rozmiar pakietu

```
// Sprawdź MTU przed wysyłaniem dużych danych
let mtu =
        peripheral.maximumWriteValueLength(for: .withoutResponse) //
        typowo 182–512 bytes
let mtuWithResponse =
        peripheral.maximumWriteValueLength(for: .withResponse) //
        512 bytes

// Dla dużych danych — fragmentuj samodzielnie
func sendLargeData(_ data: Data, to characteristic:
        CBCharacteristic) {
    let mtu =
        peripheral.maximumWriteValueLength(for: .withoutResponse)
    var offset = 0
    while offset < data.count {
        let end = min(offset + mtu, data.count)
        let chunk = data[offset..<end]
        peripheral.writeValue(chunk, for: characteristic,
        type: .withoutResponse)
        offset = end
    }
}
```

---

8. CBPeripheralManager

Konfiguracja jako Peripheral

```
class PeripheralDevice: NSObject, CBPeripheralManagerDelegate {
    var peripheralManager: CBPeripheralManager!
    var transferCharacteristic: CBMutableCharacteristic!

    override init() {
        super.init()
        peripheralManager = CBPeripheralManager(delegate: self,
        queue: .main)
    }

    func peripheralManagerDidUpdateState(_ peripheral:
        CBPeripheralManager) {
        guard peripheral.state == .poweredOn else { return }
        setupService()
    }

    func setupService() {
        let charUUID = CBUUID(string: "FFE1")
        let serviceUUID = CBUUID(string: "FFE0")

        // Tworzenie charakterystyki
        transferCharacteristic = CBMutableCharacteristic(
```

```
            type: charUUID,
            properties: [.read, .write, .notify],
            value: nil, // nil = dynamiczna wartość
            permissions: [.readable, .writeable]
        )

        // Tworzenie usługi
        let service = CBMutableService(type: serviceUUID,
        primary: true)
        service.characteristics = [transferCharacteristic]

        peripheralManager.add(service)
    }

    // Callback po dodaniu usługi
    func peripheralManager(_ peripheral: CBPeripheralManager,
        didAdd service: CBService, error: Error?) {
        guard error == nil else { return }
        startAdvertising()
    }

    func startAdvertising() {
        peripheralManager.startAdvertising([
            CBAdvertisementDataServiceUUIDsKey: [CBUUID(string:
        "FFE0")],
            CBAdvertisementDataLocalNameKey: "MójPeripheral"
        ])
    }

    func peripheralManagerDidStartAdvertising(_ peripheral:
        CBPeripheralManager, error: Error?) {
        print(error == nil ? "Advertising started" : "Błąd: \
        (error!)")
    }
}
```

Obsługa zapytań Read/Write

```
// Central pyta o odczyt
func peripheralManager(
    _ peripheral: CBPeripheralManager,
    didReceiveRead request: CBATTRequest
) {
    if request.characteristic.uuid == transferCharacteristic.uuid
        {
        request.value = "Hello".data(using: .utf8)
        peripheral.respond(to: request, withResult: .success)
    } else {
        peripheral.respond(to: request,
        withResult: .attributeNotFound)
    }
}
```

```swift
// Central wysyła zapis
func peripheralManager(
    _ peripheral: CBPeripheralManager,
    didReceiveWrite requests: [CBATTRequest]
) {
    for request in requests {
        guard let data = request.value else { continue }
        print("Otrzymano: \(data)")
        peripheral.respond(to: request, withResult: .success)
    }
}

// Wysyłanie notyfikacji do subskrybentów
func sendNotification(data: Data) {
    let sent = peripheralManager.updateValue(
        data,
        for: transferCharacteristic,
        onSubscribedCentrals: nil // nil = wszystkie subskrybenty
    )
    if !sent {
        // Queue jest pełna – poczekaj na
        peripheralManagerIsReady(toUpdateSubscribers:)
    }
}

func peripheralManagerIsReady(toUpdateSubscribers peripheral:
        CBPeripheralManager) {
    // Można ponowić wysyłkę
}
```

---

9. Background Modes

Konfiguracja

W `Info.plist` i w Xcode → Target → Signing & Capabilities → Background Modes:

```
[x] Uses Bluetooth LE accessories      → bluetooth-central
[x] Acts as a Bluetooth LE accessory   → bluetooth-peripheral
```

Zachowanie w tle

Tryb	Foreground	Background
Skanowanie bez UUID	Działa	**Zablokowane**
Skanowanie z UUID	Działa	Działa (wolniej)
Połączenie/Dane	Pełna szybkość	Działa, ograniczone

Tryb	Foreground	Background
Advertising	Pełny packet	Tylko serviceUUIDs (packet skrócony)

```
// Skanowanie w tle — MUSI mieć serviceUUIDs
centralManager.scanForPeripherals(
    withServices: [CBUUID(string: "FFE0")], // NIE nil
    options: [CBCentralManagerScanOptionAllowDuplicatesKey: false]
)
```

Powiadomienia użytkownika z tła

```
centralManager = CBCentralManager(
    delegate: self,
    queue: .main,
    options: [
        CBCentralManagerOptionRestoreIdentifierKey:
        "com.app.central",
        // Pokaż alert gdy peripheral rozłączy się w tle:
        CBConnectPeripheralOptionNotifyOnDisconnectionKey: true
    ]
)
```

---

10. State Restoration

Gdy iOS zabije aplikację w tle, Core Bluetooth może ją **wznowić** gdy zajdzie zdarzenie BLE.

Inicjalizacja z Restore ID

```
// MUSI być ten sam identyfikator zawsze
centralManager = CBCentralManager(
    delegate: self,
    queue: .main,
    options: [CBCentralManagerOptionRestoreIdentifierKey:
        "com.myapp.central"]
)
```

Przywracanie stanu

```
// Wywoływane gdy iOS wznawia aplikację przez BLE
func centralManager(
    _ central: CBCentralManager,
    willRestoreState dict: [String: Any]
) {
    // Przywróć wcześniej skanowane peripheral
```

```
    if let peripherals =
        dict[CBCentralManagerRestoredStatePeripheralsKey] as?
        [CBPeripheral] {
        for peripheral in peripherals {
            peripheral.delegate = self
            restoredPeripherals.append(peripheral)
            // Sprawdź stan: peripheral.state
        == .connected / .connecting
        }
    }

    // Przywróć skanowanie
    if let scanServices =
        dict[CBCentralManagerRestoredStateScanServicesKey] as?
        [CBUUID] {
        let scanOptions =
        dict[CBCentralManagerRestoredStateScanOptionsKey] as?
        [String: Any]
        // Wznów skanowanie po powrocie do poweredOn
    }
}
```

Pełny cykl życia z State Restoration

```
func centralManagerDidUpdateState(_ central: CBCentralManager) {
    guard central.state == .poweredOn else { return }

    // Po przywróceniu stanu — sprawdź przywrócone peripheral
    for peripheral in restoredPeripherals {
        switch peripheral.state {
        case .connected:
            // Już połączony — odkryj usługi ponownie
            peripheral.discoverServices([serviceUUID])
        case .connecting:
            // Poczekaj na didConnect
            break
        case .disconnected:
            centralManager.connect(peripheral, options: nil)
        default:
            break
        }
    }
}
```

---

11. Best Practices & Gotchas

Sprawdzaj stan managera

```
// NIE rób tak — scanner crashnie jeśli state != .poweredOn
centralManager.scanForPeripherals(withServices: nil, options:
        nil) // crash!

// Rób tak
guard centralManager.state == .poweredOn else {
    pendingScan = true // zapamiętaj intent
    return
}
centralManager.scanForPeripherals(withServices: [uuid], options:
        nil)
```

Przechowuj referencje do Peripheral

```
// Peripheral jest zwalniany przez ARC jeśli go nie trzymasz
class BLEManager {
    var peripheral: CBPeripheral? // silna referencja — konieczna!

    func centralManager(_ central: CBCentralManager, didDiscover
        peripheral: CBPeripheral, ...) {
        self.peripheral = peripheral // bez tego peripheral
        zniknie
        central.connect(peripheral, options: nil)
    }
}
```

Kolejka (Queue)

```
// Używaj dedykowanej kolejki dla lepszej wydajności
let bleQueue = DispatchQueue(label: "com.app.ble",
        qos: .userInitiated)
centralManager = CBCentralManager(delegate: self, queue: bleQueue)

// Wtedy aktualizacje UI — zawsze na main queue
DispatchQueue.main.async {
    self.tableView.reloadData()
}
```

Ograniczenia iOS — pamiętaj

- **Skanowanie `nil` UUID w tle** — zawsze zablokowane
- **Advertising w tle** — packet skrócony do samych UUID usług
- **15 sekund** — maksymalny czas na wykonanie zadania BLE po wznowieniu w tle
- **Peripheral UUID jest specificzny dla iPhone'a** — ten sam peripheral będzie miał inny UUID na innym telefonie

- **iOS może opóźnić discovery** — nie polegaj na natychmiastowym wyniku skanowania
- **Nie skanuj ciągle** — używaj `stopScan()` i `allowDuplicates: false` dla oszczędności baterii

Typowe błędy

```
// BŁĄD: Ustawianie delegata po connect
centralManager.connect(peripheral, options: nil)
peripheral.delegate = self // Za późno! Może przegapić didConnect

// POPRAWNIE:
peripheral.delegate = self // Przed connect
centralManager.connect(peripheral, options: nil)

// BŁĄD: Odkrywanie wszystkich usług i charakterystyk
peripheral.discoverServices(nil) // Kosztowne — wolne
peripheral.discoverCharacteristics(nil, for: service) // Kosztowne

// POPRAWNIE: Filtruj po znanych UUID
peripheral.discoverServices([CBUUID(string: "FFE0")])
peripheral.discoverCharacteristics([CBUUID(string: "FFE1")], for:
        service)

// BŁĄD: Ignorowanie writeWithoutResponse flow control
for chunk in chunks {
    peripheral.writeValue(chunk, for: char,
        type: .withoutResponse) // Może droppować dane!
}

// POPRAWNIE: Sprawdzaj canSendWriteWithoutResponse
if peripheral.canSendWriteWithoutResponse {
    peripheral.writeValue(chunk, for: char,
        type: .withoutResponse)
} else {
    // Poczekaj na peripheralIsReady(toSendWriteWithoutResponse:)
}
```

Testowanie

- BLE **nie działa na symulatorze** — zawsze testuj na fizycznym iPhone
- Do debugowania peripheral użyj aplikacji **LightBlue** lub **nRF Connect**
- `CBCentralManagerOptionShowPowerAlertKey: true` — pokazuje systemowy alert gdy BT wyłączone

```
centralManager = CBCentralManager(
    delegate: self,
    queue: .main,
    options: [CBCentralManagerOptionShowPowerAlertKey: true]
)
```

Quick Reference

```
Inicjalizacja:  CBCentralManager(delegate:queue:options:)
Stan:           centralManagerDidUpdateState(_:)  → wymagany
Skanowanie:     scanForPeripherals(withServices:options:)
Discovery:      didDiscover(peripheral:advertisementData:rssi:)
Połączenie:     connect(_:options:) → didConnect → setDelegate →
discoverServices
Services:       discoverServices([uuid]) → didDiscoverServices
Chars:          discoverCharacteristics([uuid], for:) →
didDiscoverCharacteristics
Odczyt:         readValue(for:) → didUpdateValueFor
Zapis:          writeValue(_:for:type:) → didWriteValueFor
(tylko .withResponse)
Notify:         setNotifyValue(true, for:) → didUpdateValueFor
(każda zmiana)
Rozłączenie:    cancelPeripheralConnection(_:) →
didDisconnectPeripheral
```